



How the resume site works

A career timeline that tailors itself to each role, then hands you a matching resume.

christianschlaefer.com
Christian Schlaefer

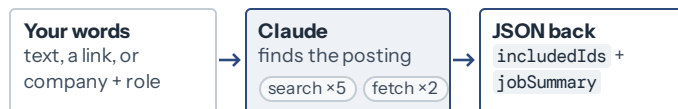
The idea

Instead of one static resume, the site lays my career out on a **proportional timeline**: every role spans its real dates, side projects branch off as tags, and skills level up as you scroll. Tell it about a role and an LLM (**Claude**) curates what you see, then compiles a two-page resume from the same data, tailoring my own words without adding anything I didn't write.



1 Reading the role

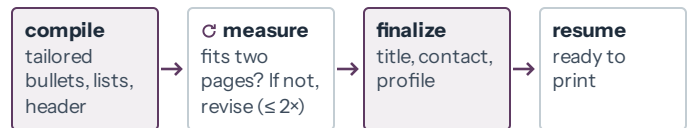
Visitors can enter a full description, a link, or just a company and title. `/api/curate-timeline` hands it to Claude with **web search** and **web fetch**: a link gets read, a partial entry becomes a search for the real posting, and if nothing turns up, Claude works from what was typed.



Experiences always stay; only Points get judged. The summary carries Claude's findings forward, and if anything fails, everything shows.

2 Two calls build the resume

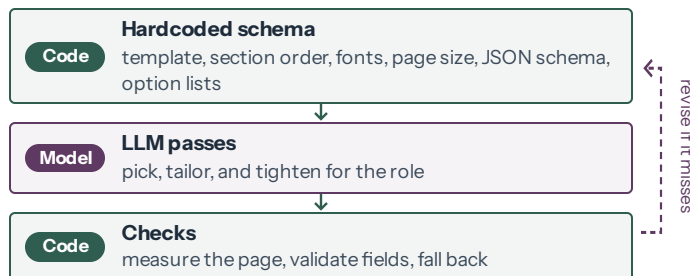
At the end of the timeline, the browser runs two calls back to back. `compile-resume` builds the body; `finalize-resume` writes the header, but only after the body is final.



Measuring happens in an invisible, page-sized clone in the browser. The header comes last so the Executive Profile only summarizes what made the page; if it would break the fit, my baseline profile steps in.

The formatting scheme

Reliable formatting comes from splitting the work: fixed rules are **hardcoded**, judgment goes to the **model**, and code checks the result, sending misses back for another pass.



Each guardrail on the right puts one of these ideas into practice.

Keeping the AI honest

- Tailors, never invents.** The model may combine or reword an entry's bullets for the role and the page, but only from that entry's own text: no new facts, numbers, or skills.
- Closed lists.** Headers, expertise, fluency, title, and contact line are enums in a forced tool call; nothing new can be typed.
- Code double-checks.** The server re-validates every field and keeps at least one bullet per recent role.
- Skills stay home.** Stripped before every API call, so a rewrite has no inferred skill to borrow.
- A shorter leash on the profile.** It's written only from the finished resume, capped at 130% of my baseline.
- Fallbacks everywhere.** If a call fails, a mechanical resume still renders.

3 Getting in touch

Seven seconds into the resume, a popup offers to connect. Submitting hits `/api/submit-lead`, which checks the request, then signs in to Gmail's SMTP server through Nodemailer.

Keys stay on the server

The Claude API key and Google App Password live only in Vercel environment variables; the browser never sees either.

Rate limits

Vercel's firewall allows each visitor 10 API calls a minute; the email function adds 3 per 10 minutes and one thank-you per address a day.

Guarded functions

Every function turns away other sites, oversized payloads, and over-long input; fetched pages count as data, never instructions; a honeypot catches bots.